

BASE DE DATOS II

Práctica de Laboratorio y Análisis

Fragmentación Vertical y Horizontal Procesamiento de Consultas en Bases de Datos Distribuidas

Descomposición | Localización | Optimización | PostgreSQL

Campo	Descripción
Curso	Base de Datos II
Nivel	Universitario
Duración sugerida	2 a 3 horas
Modalidad	Individual o grupal
Herramienta	PostgreSQL
Entregable	Informe técnico con evidencias de ejecución y análisis

Sesión 8, Fragmentación Vertical y Horizontal.

1. Presentación de la práctica

La presente práctica tiene como finalidad que el estudiante aplique los fundamentos de la fragmentación horizontal y vertical en bases de datos distribuidas, utilizando PostgreSQL como motor de base de datos. El laboratorio integra actividades de implementación SQL, análisis técnico y propuesta de diseño distribuido.

El desarrollo simula un escenario empresarial donde los datos de clientes, empleados y ventas deben ser descompuestos en fragmentos para mejorar el rendimiento, reducir tráfico entre nodos, facilitar la localización de datos y optimizar el procesamiento de consultas.

Importante: Aunque PostgreSQL se ejecutará en un entorno local, la práctica simula el comportamiento de una base de datos distribuida mediante tablas fragmentadas y una tabla de localización de fragmentos.

2. Logro de aprendizaje

Al finalizar la práctica, el estudiante aplica conceptos de bases de datos distribuidas para gestionar adecuadamente la información mediante fragmentación, localización y optimización de consultas.

3. Competencias a desarrollar

- Diferenciar fragmentación horizontal y fragmentación vertical.
- Aplicar criterios de descomposición según patrones de consulta.
- Simular la localización física de fragmentos en distintos nodos.
- Diseñar consultas optimizadas sobre datos fragmentados.
- Interpretar planes de ejecución con EXPLAIN y EXPLAIN ANALYZE.
- Justificar técnicamente una estrategia de fragmentación para un escenario empresarial.

4. Caso de estudio

La empresa Comercial Andina S.A.C. administra clientes, empleados y ventas en diferentes regiones del país: Norte, Sur, Centro y Oriente. Actualmente, la información está centralizada, lo que genera problemas de rendimiento cuando las sedes regionales consultan grandes volúmenes de datos. La empresa desea aplicar estrategias de fragmentación para mejorar el acceso, reducir la latencia y optimizar consultas.

5. Parte I: Preparación de la base de datos

5.1 Creación de la base de datos

```
CREATE DATABASE bd_distribuida_practica;

-- Conectarse desde psql:
\c bd_distribuida_practica;
```

5.2 Creación de tablas base

```
CREATE TABLE clientes (
    id_cliente SERIAL PRIMARY KEY,
    nombres VARCHAR(100) NOT NULL,
    apellidos VARCHAR(100) NOT NULL,
    dni CHAR(8) UNIQUE NOT NULL,
```

```

    region VARCHAR(20) NOT NULL,
    ciudad VARCHAR(50) NOT NULL,
    direccion VARCHAR(150),
    telefono VARCHAR(20),
    correo VARCHAR(100),
    fecha_registro DATE NOT NULL
);

CREATE TABLE empleados (
    id_empleado SERIAL PRIMARY KEY,
    nombres VARCHAR(100) NOT NULL,
    apellidos VARCHAR(100) NOT NULL,
    cargo VARCHAR(50) NOT NULL,
    salario NUMERIC(10,2) NOT NULL,
    region VARCHAR(20) NOT NULL,
    fecha_contratacion DATE NOT NULL,
    correo VARCHAR(100),
    telefono VARCHAR(20)
);

CREATE TABLE ventas (
    id_venta SERIAL PRIMARY KEY,
    id_cliente INT NOT NULL,
    id_empleado INT NOT NULL,
    fecha_venta DATE NOT NULL,
    monto_total NUMERIC(10,2) NOT NULL,
    region VARCHAR(20) NOT NULL,
    estado VARCHAR(20) NOT NULL,
    FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente),
    FOREIGN KEY (id_empleado) REFERENCES empleados(id_empleado)
);

```

5.3 Inserción de datos de prueba

```

INSERT INTO clientes
(nombres, apellidos, dni, region, ciudad, direccion, telefono, correo, fecha_registro)
VALUES
('Luis', 'Ramírez', '12345678', 'Norte', 'Trujillo', 'Av. España 123', '987654321',
'luis.ramirez@mail.com', '2024-01-10'),
('María', 'Torres', '23456789', 'Sur', 'Arequipa', 'Calle Mercaderes 456', '987111222',
'maria.torres@mail.com', '2024-02-15'),
('Carlos', 'Mendoza', '34567890', 'Centro', 'Huancayo', 'Av. Real 789', '987333444',
'carlos.mendoza@mail.com', '2024-03-20'),
('Ana', 'Paredes', '45678901', 'Oriente', 'Iquitos', 'Jr. Amazonas 321', '987555666',
'ana.paredes@mail.com', '2024-04-05'),
('Jorge', 'Castillo', '56789012', 'Norte', 'Chiclayo', 'Av. Balta 654', '987777888',
'jorge.castillo@mail.com', '2024-05-12'),
('Rosa', 'Vargas', '67890123', 'Sur', 'Tacna', 'Av. Bolognesi 987', '987999000', 'rosa.vargas@mail.com',
'2024-06-18');

INSERT INTO empleados
(nombres, apellidos, cargo, salario, region, fecha_contratacion, correo, telefono)
VALUES
('Pedro', 'Salazar', 'Vendedor', 2500.00, 'Norte', '2022-01-15', 'pedro.salazar@empresa.com',
'999111222'),
('Lucía', 'Fernández', 'Supervisora', 4200.00, 'Sur', '2021-07-10', 'lucia.fernandez@empresa.com',
'999333444'),
('Miguel', 'Reyes', 'Vendedor', 2600.00, 'Centro', '2023-03-05', 'miguel.reyes@empresa.com',
'999555666'),
('Elena', 'Campos', 'Vendedora', 2550.00, 'Oriente', '2023-09-12', 'elena.campos@empresa.com',
'999777888');

INSERT INTO ventas
(id_cliente, id_empleado, fecha_venta, monto_total, region, estado)
VALUES
(1, 1, '2024-07-01', 1500.00, 'Norte', 'Completada'),
(2, 2, '2024-07-03', 2300.00, 'Sur', 'Completada'),
(3, 3, '2024-07-05', 900.00, 'Centro', 'Pendiente'),

```

```
(4, 4, '2024-07-07', 3100.00, 'Oriente', 'Completada'),
(5, 1, '2024-07-10', 1800.00, 'Norte', 'Completada'),
(6, 2, '2024-07-12', 2700.00, 'Sur', 'Pendiente');
```

6. Parte II: Fragmentación horizontal

La fragmentación horizontal divide una tabla en subconjuntos de filas. Cada fragmento conserva la misma estructura de columnas, pero contiene registros diferentes según un criterio.

6.1 Fragmentación horizontal de clientes por región

```
CREATE TABLE clientes_norte AS
SELECT * FROM clientes
WHERE region = 'Norte';

CREATE TABLE clientes_sur AS
SELECT * FROM clientes
WHERE region = 'Sur';

CREATE TABLE clientes_centro AS
SELECT * FROM clientes
WHERE region = 'Centro';

CREATE TABLE clientes_oriente AS
SELECT * FROM clientes
WHERE region = 'Oriente';
```

6.2 Verificación de fragmentos

```
SELECT * FROM clientes_norte;
SELECT * FROM clientes_sur;
SELECT * FROM clientes_centro;
SELECT * FROM clientes_oriente;
```

6.3 Reconstrucción lógica de la tabla clientes

```
SELECT * FROM clientes_norte
UNION ALL
SELECT * FROM clientes_sur
UNION ALL
SELECT * FROM clientes_centro
UNION ALL
SELECT * FROM clientes_oriente;
```

6.4 Actividades de análisis

1. ¿Qué criterio se utilizó para aplicar la fragmentación horizontal?
2. ¿Por qué la región es un atributo adecuado para fragmentar la tabla clientes?
3. ¿Qué ventajas tendría esta fragmentación si cada región tuviera su propio servidor?
4. ¿Qué problemas podrían aparecer si un cliente cambia de región?
5. Explique por qué UNION ALL puede ser más adecuado que UNION para reconstruir los fragmentos.

7. Parte III: Fragmentación vertical

La fragmentación vertical divide una tabla por columnas. Cada fragmento contiene un subconjunto de atributos y debe conservar la clave primaria para permitir la reconstrucción de la relación original.

7.1 Fragmentación vertical de empleados

```
CREATE TABLE empleados_datos_basicos AS
SELECT
    id_empleado,
    nombres,
    apellidos,
    cargo,
    region
FROM empleados;

CREATE TABLE empleados_datos_confidenciales AS
SELECT
    id_empleado,
    salario,
    fecha_contratacion,
    correo,
    telefono
FROM empleados;
```

7.2 Verificación de fragmentos verticales

```
SELECT * FROM empleados_datos_basicos;
SELECT * FROM empleados_datos_confidenciales;
```

7.3 Reconstrucción de empleados mediante JOIN

```
SELECT
    b.id_empleado,
    b.nombres,
    b.apellidos,
    b.cargo,
    c.salario,
    b.region,
    c.fecha_contratacion,
    c.correo,
    c.telefono
FROM empleados_datos_basicos b
INNER JOIN empleados_datos_confidenciales c
ON b.id_empleado = c.id_empleado;
```

7.4 Actividades de análisis

6. ¿Qué atributos se colocaron en el fragmento de datos básicos?
7. ¿Qué atributos se colocaron en el fragmento confidencial?
8. ¿Por qué es obligatorio conservar id_empleado en ambos fragmentos?
9. ¿Qué ventaja ofrece esta fragmentación desde el punto de vista de seguridad?
10. ¿Qué ventaja ofrece esta fragmentación desde el punto de vista de rendimiento?
11. ¿Qué desventaja puede aparecer al reconstruir la tabla original?

8. Parte IV: Fragmentación horizontal de ventas

8.1 Crear fragmentos por estado de venta

```
CREATE TABLE ventas_completadas AS
SELECT * FROM ventas
WHERE estado = 'Completada';

CREATE TABLE ventas_pendientes AS
SELECT * FROM ventas
WHERE estado = 'Pendiente';
```

8.2 Comparación de consultas

```
-- Consulta sobre la tabla original
SELECT *
FROM ventas
WHERE estado = 'Completada';

-- Consulta sobre el fragmento correspondiente
SELECT *
FROM ventas_completadas;
```

8.3 Actividad de análisis

12. ¿Cuál de las dos consultas podría ser más eficiente en un escenario distribuido?
13. ¿Qué fragmento evita consultar datos no necesarios?
14. ¿Cómo impacta esta fragmentación en el tráfico entre nodos?
15. ¿Qué desventaja tendría fragmentar ventas solo por estado?

9. Parte V: Simulación de localización de fragmentos

La localización indica en qué nodo físico o lógico se almacena cada fragmento. Esta información permite seleccionar los fragmentos relevantes para una consulta y reducir transferencias innecesarias.

```
CREATE TABLE localizacion_fragmentos (
    id_localizacion SERIAL PRIMARY KEY,
    nombre_fragmento VARCHAR(100) NOT NULL,
    tabla_origen VARCHAR(100) NOT NULL,
    tipo_fragmentacion VARCHAR(30) NOT NULL,
    criterio_fragmentacion VARCHAR(100),
    nodo_asignado VARCHAR(100) NOT NULL,
    ubicacion_geografica VARCHAR(100) NOT NULL
);

INSERT INTO localizacion_fragmentos
(nombre_fragmento, tabla_origen, tipo_fragmentacion, criterio_fragmentacion, nodo_asignado,
ubicacion_geografica)
VALUES
('clientes_norte', 'clientes', 'Horizontal', 'region = Norte', 'Nodo_01', 'Trujillo'),
('clientes_sur', 'clientes', 'Horizontal', 'region = Sur', 'Nodo_02', 'Arequipa'),
('clientes_centro', 'clientes', 'Horizontal', 'region = Centro', 'Nodo_03', 'Huancayo'),
('clientes_oriente', 'clientes', 'Horizontal', 'region = Oriente', 'Nodo_04', 'Iquitos'),
('empleados_datos_basicos', 'empleados', 'Vertical', 'datos frecuentes', 'Nodo_01', 'Lima'),
('empleados_datos_confidenciales', 'empleados', 'Vertical', 'datos sensibles', 'Nodo_Seguro', 'Lima'),
('ventas_completadas', 'ventas', 'Horizontal', 'estado = Completada', 'Nodo_Analitico', 'Lima'),
('ventas_pendientes', 'ventas', 'Horizontal', 'estado = Pendiente', 'Nodo_Operativo', 'Lima');
```

9.1 Consultas de localización

```
SELECT *
FROM localizacion_fragmentos;

SELECT
    nombre_fragmento,
    tipo_fragmentacion,
    criterio_fragmentacion,
    nodo_asignado,
    ubicacion_geografica
FROM localizacion_fragmentos
WHERE tabla_origen = 'clientes';
```

9.2 Actividades de análisis

16. ¿Qué importancia tiene conocer la localización de los fragmentos?

17. ¿Cómo ayuda la localización a reducir la latencia?
18. ¿Qué fragmentos deberían estar cerca de los usuarios regionales?
19. ¿Qué fragmentos deberían estar en nodos más seguros?
20. ¿Qué problemas podrían ocurrir si la localización de fragmentos no está documentada?

10. Parte VI: Optimización de consultas

10.1 Creación de índices

```
CREATE INDEX idx_clientes_region ON clientes(region);
CREATE INDEX idx_ventas_estado ON ventas(estado);
CREATE INDEX idx_ventas_region ON ventas(region);
CREATE INDEX idx_empleados_region ON empleados(region);

-- Actualizar estadísticas para mejorar las estimaciones del optimizador
ANALYZE clientes;
ANALYZE empleados;
ANALYZE ventas;
```

10.2 Análisis con EXPLAIN

```
EXPLAIN
SELECT *
FROM clientes
WHERE region = 'Norte';

EXPLAIN
SELECT *
FROM clientes_norte;
```

10.3 Análisis con EXPLAIN ANALYZE

```
EXPLAIN ANALYZE
SELECT *
FROM ventas
WHERE estado = 'Completada';

EXPLAIN ANALYZE
SELECT *
FROM ventas_completadas;
```

10.4 Actividades de análisis técnico

21. ¿Qué diferencias observa entre consultar la tabla original y consultar el fragmento?
22. ¿PostgreSQL utiliza índices en la consulta sobre la tabla original?
23. ¿La consulta sobre el fragmento reduce el número de filas evaluadas?
24. ¿Qué consulta presenta menor costo estimado?
25. ¿Qué consulta presenta menor tiempo real de ejecución?
26. ¿Cómo cambiarían los resultados si la tabla tuviera millones de registros?

11. Parte VII: Consulta distribuida simulada

11.1 Consulta con varios fragmentos

```
SELECT
  v.id_venta,
  c.nombres AS nombre_cliente,
  c.apellidos AS apellido_cliente,
```

```

    e.nombres AS nombre_empleado,
    e.apellidos AS apellido_empleado,
    v.fecha_venta,
    v.monto_total,
    v.region
FROM ventas_completadas v
INNER JOIN clientes c
ON v.id_cliente = c.id_cliente
INNER JOIN empleados_datos_basicos e
ON v.id_empleado = e.id_empleado;

```

11.2 Consulta optimizada para la región Norte

```

SELECT
    v.id_venta,
    c.nombres AS nombre_cliente,
    c.apellidos AS apellido_cliente,
    e.nombres AS nombre_empleado,
    e.apellidos AS apellido_empleado,
    v.fecha_venta,
    v.monto_total,
    v.region
FROM ventas_completadas v
INNER JOIN clientes_norte c
ON v.id_cliente = c.id_cliente
INNER JOIN empleados_datos_basicos e
ON v.id_empleado = e.id_empleado
WHERE v.region = 'Norte';

```

11.3 Actividades de análisis

27. ¿Por qué la segunda consulta puede ser más eficiente?
28. ¿Qué fragmentos intervienen en la consulta optimizada?
29. ¿Qué fragmentos se evitan consultar?
30. ¿Cómo se reduce el tráfico entre nodos?
31. ¿Qué estrategia adicional aplicaría para optimizar esta consulta?

12. Parte VIII: Diseño de fragmentación propio

La empresa espera manejar 2 millones de clientes, 500 mil empleados históricos y 20 millones de ventas. Además, existen consultas frecuentes por región, reportes financieros mensuales, acceso restringido a datos salariales y usuarios regionales que consultan principalmente su propia zona.

12.1 Tabla de propuesta de fragmentación

Tabla	Tipo de fragmentación	Criterio	Fragmentos propuestos	Justificación
clientes				
empleados				
ventas				

12.2 Preguntas de análisis avanzado

32. ¿Aplicaría fragmentación horizontal, vertical o mixta en la tabla clientes?
33. ¿Qué atributos de empleados deberían estar protegidos en un fragmento separado?
34. ¿La tabla ventas debería fragmentarse por región, fecha, estado o una combinación?
35. ¿Qué impacto tendría la fragmentación en operaciones INSERT, UPDATE y DELETE?
36. ¿Cómo garantizaría que los fragmentos permitan reconstruir las tablas originales?

37. ¿Qué riesgos aparecen cuando los fragmentos están distribuidos en diferentes ubicaciones geográficas?
38. ¿Cómo influye la localización de datos en el tiempo de respuesta?
39. ¿Qué criterios usaría para decidir en qué nodo ubicar cada fragmento?

13. Entregable final

El estudiante deberá presentar un informe técnico con la siguiente estructura:

- Carátula: nombre del estudiante, curso, docente, fecha y título de la práctica.
- Introducción: bases de datos distribuidas e importancia de la fragmentación.
- Desarrollo en PostgreSQL: creación de tablas, inserción de datos, fragmentación, reconstrucción y consultas.
- Evidencias: capturas de ejecución y resultados de consultas.
- Análisis: respuestas a las preguntas planteadas e interpretación de EXPLAIN ANALYZE.
- Propuesta de diseño: estrategia de fragmentación y localización.
- Conclusiones: mínimo tres conclusiones relacionadas con fragmentación, localización y optimización.
- Anexos: código SQL completo y evidencias adicionales.

14. Rúbrica de evaluación

Criterio	Excelente (4 pts)	Bueno (3 pts)	Regular (2 pts)	Deficiente (1 pt)
Comprensión conceptual	Explica con claridad fragmentación horizontal, vertical, localización y optimización.	Explica la mayoría de conceptos correctamente.	Presenta explicaciones incompletas.	Presenta errores conceptuales graves.
Implementación en PostgreSQL	Crea correctamente tablas, fragmentos, consultas e índices.	Presenta pequeños errores corregibles.	Implementación parcial.	No logra implementar adecuadamente.
Análisis de resultados	Interpreta costos, tiempos y efectos de la fragmentación.	Analiza resultados de forma general.	Análisis superficial.	No analiza resultados.
Diseño de fragmentación	Propone una estrategia coherente, justificada y escalable.	Propone una estrategia aceptable.	La propuesta es incompleta.	La propuesta no es viable.
Informe final	Presentación profesional, ordenada y completa.	Presentación clara con mínimos errores.	Presentación desordenada o incompleta.	No cumple estructura solicitada.

15. Preguntas de cierre

40. ¿Qué diferencia principal existe entre fragmentación horizontal y vertical?
41. ¿Por qué la fragmentación puede mejorar el rendimiento?
42. ¿Qué papel cumple la descomposición en una base de datos distribuida?
43. ¿Qué significa localizar un fragmento?

44. ¿Por qué no siempre conviene fragmentar una tabla?
45. ¿Qué relación existe entre fragmentación y optimización de consultas?
46. ¿Qué aprendió al aplicar fragmentación usando PostgreSQL?

Anexo A: Script SQL completo

El siguiente script integra la preparación de la base de datos, la creación de tablas, la inserción de datos, la fragmentación horizontal y vertical, la localización y las consultas de análisis.

```
-- =====
-- PRACTICA: FRAGMENTACION VERTICAL Y HORIZONTAL EN POSTGRESQL
-- CURSO: BASE DE DATOS II
-- =====

CREATE DATABASE bd_distribuida_practica;

-- Conectarse desde psql:
-- \c bd_distribuida_practica;

-- =====
-- 1. TABLAS BASE
-- =====

CREATE TABLE clientes (
    id_cliente SERIAL PRIMARY KEY,
    nombres VARCHAR(100) NOT NULL,
    apellidos VARCHAR(100) NOT NULL,
    dni CHAR(8) UNIQUE NOT NULL,
    region VARCHAR(20) NOT NULL,
    ciudad VARCHAR(50) NOT NULL,
    direccion VARCHAR(150),
    telefono VARCHAR(20),
    correo VARCHAR(100),
    fecha_registro DATE NOT NULL
);

CREATE TABLE empleados (
    id_empleado SERIAL PRIMARY KEY,
    nombres VARCHAR(100) NOT NULL,
    apellidos VARCHAR(100) NOT NULL,
    cargo VARCHAR(50) NOT NULL,
    salario NUMERIC(10,2) NOT NULL,
    region VARCHAR(20) NOT NULL,
    fecha_contratacion DATE NOT NULL,
    correo VARCHAR(100),
    telefono VARCHAR(20)
);

CREATE TABLE ventas (
    id_venta SERIAL PRIMARY KEY,
    id_cliente INT NOT NULL,
    id_empleado INT NOT NULL,
    fecha_venta DATE NOT NULL,
```

```

    monto_total NUMERIC(10,2) NOT NULL,
    region VARCHAR(20) NOT NULL,
    estado VARCHAR(20) NOT NULL,
    FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente),
    FOREIGN KEY (id_empleado) REFERENCES empleados(id_empleado)
);

-- =====
-- 2. DATOS DE PRUEBA
-- =====

INSERT INTO clientes
(nombres, apellidos, dni, region, ciudad, direccion, telefono, correo, fecha_registro)
VALUES
('Luis', 'Ramírez', '12345678', 'Norte', 'Trujillo', 'Av. España 123', '987654321', 'luis.ramirez@mail.com', '2024-01-10'),
('María', 'Torres', '23456789', 'Sur', 'Arequipa', 'Calle Mercaderes 456', '987111222', 'maria.torres@mail.com', '2024-02-15'),
('Carlos', 'Mendoza', '34567890', 'Centro', 'Huancayo', 'Av. Real 789', '987333444', 'carlos.mendoza@mail.com', '2024-03-20'),
('Ana', 'Paredes', '45678901', 'Oriente', 'Iquitos', 'Jr. Amazonas 321', '987555666', 'ana.paredes@mail.com', '2024-04-05'),
('Jorge', 'Castillo', '56789012', 'Norte', 'Chiclayo', 'Av. Balta 654', '987777888', 'jorge.castillo@mail.com', '2024-05-12'),
('Rosa', 'Vargas', '67890123', 'Sur', 'Tacna', 'Av. Bolognesi 987', '987999000', 'rosa.vargas@mail.com', '2024-06-18');

INSERT INTO empleados
(nombres, apellidos, cargo, salario, region, fecha_contratacion, correo, telefono)
VALUES
('Pedro', 'Salazar', 'Vendedor', 2500.00, 'Norte', '2022-01-15', 'pedro.salazar@empresa.com', '999111222'),
('Lucía', 'Fernández', 'Supervisora', 4200.00, 'Sur', '2021-07-10', 'lucia.fernandez@empresa.com', '999333444'),
('Miguel', 'Reyes', 'Vendedor', 2600.00, 'Centro', '2023-03-05', 'miguel.reyes@empresa.com', '999555666'),
('Elena', 'Campos', 'Vendedora', 2550.00, 'Oriente', '2023-09-12', 'elena.campos@empresa.com', '999777888');

INSERT INTO ventas
(id_cliente, id_empleado, fecha_venta, monto_total, region, estado)
VALUES
(1, 1, '2024-07-01', 1500.00, 'Norte', 'Completada'),
(2, 2, '2024-07-03', 2300.00, 'Sur', 'Completada'),
(3, 3, '2024-07-05', 900.00, 'Centro', 'Pendiente'),
(4, 4, '2024-07-07', 3100.00, 'Oriente', 'Completada'),
(5, 1, '2024-07-10', 1800.00, 'Norte', 'Completada'),
(6, 2, '2024-07-12', 2700.00, 'Sur', 'Pendiente');

-- =====
-- 3. FRAGMENTACION HORIZONTAL: CLIENTES POR REGION
-- =====

CREATE TABLE clientes_norte AS SELECT * FROM clientes WHERE region = 'Norte';
CREATE TABLE clientes_sur AS SELECT * FROM clientes WHERE region = 'Sur';
CREATE TABLE clientes_centro AS SELECT * FROM clientes WHERE region = 'Centro';
CREATE TABLE clientes_oriente AS SELECT * FROM clientes WHERE region = 'Oriente';

SELECT * FROM clientes_norte;
SELECT * FROM clientes_sur;
SELECT * FROM clientes_centro;
SELECT * FROM clientes_oriente;

```

```
-- Reconstruccion logica
SELECT * FROM clientes_norte
UNION ALL
SELECT * FROM clientes_sur
UNION ALL
SELECT * FROM clientes_centro
UNION ALL
SELECT * FROM clientes_oriente;

-- =====
-- 4. FRAGMENTACION VERTICAL: EMPLEADOS
-- =====

CREATE TABLE empleados_datos_basicos AS
SELECT id_empleado, nombres, apellidos, cargo, region
FROM empleados;

CREATE TABLE empleados_datos_confidenciales AS
SELECT id_empleado, salario, fecha_contratacion, correo, telefono
FROM empleados;

SELECT * FROM empleados_datos_basicos;
SELECT * FROM empleados_datos_confidenciales;

-- Reconstruccion mediante JOIN
SELECT
    b.id_empleado,
    b.nombres,
    b.apellidos,
    b.cargo,
    b.salario,
    b.region,
    c.fecha_contratacion,
    c.correo,
    c.telefono
FROM empleados_datos_basicos b
INNER JOIN empleados_datos_confidenciales c
ON b.id_empleado = c.id_empleado;

-- =====
-- 5. FRAGMENTACION HORIZONTAL: VENTAS POR ESTADO
-- =====

CREATE TABLE ventas_completadas AS
SELECT * FROM ventas
WHERE estado = 'Completada';

CREATE TABLE ventas_pendientes AS
SELECT * FROM ventas
WHERE estado = 'Pendiente';
```

```

SELECT * FROM ventas_completadas;
SELECT * FROM ventas_pendientes;

-- Comparacion de consultas
SELECT * FROM ventas WHERE estado = 'Completada';
SELECT * FROM ventas_completadas;

-- =====
-- 6. LOCALIZACION DE FRAGMENTOS
-- =====

CREATE TABLE localizacion_fragmentos (
    id_localizacion SERIAL PRIMARY KEY,
    nombre_fragmento VARCHAR(100) NOT NULL,
    tabla_origen VARCHAR(100) NOT NULL,
    tipo_fragmentacion VARCHAR(30) NOT NULL,
    criterio_fragmentacion VARCHAR(100),
    nodo_asignado VARCHAR(100) NOT NULL,
    ubicacion_geografica VARCHAR(100) NOT NULL
);

INSERT INTO localizacion_fragmentos
(nombre_fragmento, tabla_origen, tipo_fragmentacion, criterio_fragmentacion, nodo_asignado, ubicacion_geografica)
VALUES
('clientes_norte', 'clientes', 'Horizontal', 'region = Norte', 'Nodo_01', 'Trujillo'),
('clientes_sur', 'clientes', 'Horizontal', 'region = Sur', 'Nodo_02', 'Arequipa'),
('clientes_centro', 'clientes', 'Horizontal', 'region = Centro', 'Nodo_03', 'Huancayo'),
('clientes_oriente', 'clientes', 'Horizontal', 'region = Oriente', 'Nodo_04', 'Iquitos'),
('empleados_datos_basicos', 'empleados', 'Vertical', 'datos frecuentes', 'Nodo_01', 'Lima'),
('empleados_datos_confidenciales', 'empleados', 'Vertical', 'datos sensibles', 'Nodo_Seguro', 'Lima'),
('ventas_completadas', 'ventas', 'Horizontal', 'estado = Completada', 'Nodo_Analitico', 'Lima'),
('ventas_pendientes', 'ventas', 'Horizontal', 'estado = Pendiente', 'Nodo_Operativo', 'Lima');

SELECT * FROM localizacion_fragmentos;

SELECT nombre_fragmento, tipo_fragmentacion, criterio_fragmentacion, nodo_asignado, ubicacion_geografica
FROM localizacion_fragmentos
WHERE tabla_origen = 'clientes';

-- =====
-- 7. OPTIMIZACION E INDICES
-- =====

CREATE INDEX idx_clientes_region ON clientes(region);
CREATE INDEX idx_ventas_estado ON ventas(estado);
CREATE INDEX idx_ventas_region ON ventas(region);
CREATE INDEX idx_empleados_region ON empleados(region);

ANALYZE clientes;
ANALYZE empleados;
ANALYZE ventas;

```

```

EXPLAIN
SELECT * FROM clientes WHERE region = 'Norte';

EXPLAIN
SELECT * FROM clientes_norte;

EXPLAIN ANALYZE
SELECT * FROM ventas WHERE estado = 'Completada';

EXPLAIN ANALYZE
SELECT * FROM ventas_completadas;

-- =====
-- 8. CONSULTA DISTRIBUIDA SIMULADA
-- =====

SELECT
    v.id_venta,
    c.nombres AS nombre_cliente,
    c.apellidos AS apellido_cliente,
    e.nombres AS nombre_empleado,
    e.apellidos AS apellido_empleado,
    v.fecha_venta,
    v.monto_total,
    v.region
FROM ventas_completadas v
INNER JOIN clientes c
ON v.id_cliente = c.id_cliente
INNER JOIN empleados_datos_basicos e
ON v.id_empleado = e.id_empleado;

-- Consulta optimizada para region Norte
SELECT
    v.id_venta,
    c.nombres AS nombre_cliente,
    c.apellidos AS apellido_cliente,
    e.nombres AS nombre_empleado,
    e.apellidos AS apellido_empleado,
    v.fecha_venta,
    v.monto_total,
    v.region
FROM ventas_completadas v
INNER JOIN clientes_norte c
ON v.id_cliente = c.id_cliente
INNER JOIN empleados_datos_basicos e
ON v.id_empleado = e.id_empleado
WHERE v.region = 'Norte';

```